

Отчёт по лабораторной работе №5

Расширенная настройка HTTP-сервера Apache

Агаева Зейнаб Фуад кызы

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
3	Вывод	18
4	Контрольные вопросы	20

Список иллюстраций

2.1	Генерация закрытого ключа и самоподписанного сертификата для веб-сервера	7
2.2	Конфигурация виртуального хоста <code>www.zfagaeva.net</code> для автоматического перехода с HTTP на HTTPS	9
2.3	Разрешение HTTPS в межсетевом экране, перезапуск и проверка Apache HTTP Server	10
2.4	Предупреждение Firefox при первом подключении к серверу с самоподписанным сертификатом	11
2.5	Открытие веб-сервера <code>www.zfagaeva.net</code> после перехода на HTTPS	11
2.6	Просмотр сведений о самоподписанном сертификате веб-сервера	12
2.7	Содержимое файла <code>index.php</code> с вызовом функции <code>phpinfo</code>	13
2.8	Установка PHP, настройка прав доступа, восстановление контекстов SELinux и перезапуск Apache	14
2.9	Проверка работы PHP на веб-сервере с помощью функции <code>phpinfo</code>	15
2.10	Скорректированный provisioning-скрипт <code>http.sh</code> с установкой PHP и настройкой HTTP/HTTPS	17

Список таблиц

1 Цель работы

Приобретение практических навыков по расширенному конфигурированию HTTP-сервера Apache в части безопасности и возможности использования PHP.

2 Выполнение лабораторной работы

На виртуальной машине `server` перехожу в режим суперпользователя и выполняю подготовку каталогов для хранения закрытого ключа и сертификата веб-сервера. В каталоге `/etc/pki/tls` создаю каталог `private`, а для совместимости с используемыми в конфигурации Apache путями создаю символическую ссылку `/etc/ssl/private` на `/etc/pki/tls/private`. После этого перехожу в каталог с закрытыми ключами и генерирую новый закрытый RSA-ключ длиной 2048 бит и самоподписанный сертификат командой `openssl req -x509 -nodes -newkey rsa:2048 -keyout www.zfagaeva.net.key -out www.zfagaeva.net.crt` (см. рис. Рисунок 2.1).

Параметр `req` запускает обработку запроса на создание сертификата, а параметр `-x509` указывает OpenSSL непосредственно создать самоподписанный сертификат формата X.509. Параметр `-nodes` отключает шифрование закрытого ключа парольной фразой, что позволяет службе Apache использовать ключ автоматически при запуске без ручного ввода пароля. Параметр `-newkey rsa:2048` задаёт одновременное создание нового закрытого RSA-ключа длиной 2048 бит. С помощью `-keyout www.zfagaeva.net.key` задаю имя файла закрытого ключа, а параметром `-out www.zfagaeva.net.crt` — имя создаваемого сертификата.

При заполнении данных сертификата в поле страны `Country Name` указываю `RU`, в поле региона `State or Province Name` — `Russia`, в поле города `Locality Name` — `Moscow`. В качестве организации и подразделения указываю `zfagaeva`. В поле `Common Name` задаю доменное имя `zfagaeva.net`, а в поле адреса

электронной почты — `zfagaeva@zfagaeva.net`. После завершения генерации проверяю наличие файлов командой `ls`. В каталоге присутствуют сертификат `www.zfagaeva.net.crt` и закрытый ключ `www.zfagaeva.net.key`. Сертификат копирую в каталог `/etc/ssl/certs/`, откуда Apache будет использовать его при установлении HTTPS-соединения.

```
[root@server.zfagaeva.net private]# openssl req -x509 -nodes -newkey rsa:2048 -keyout www.zfagaeva.net.key -out www.zfagaeva.net.crt
-----
Country Name (2 letter code) [XX]:RU
State or Province Name (full name) []:Russia
Locality Name (eg, city) [Default City]:Moscow
Organization Name (eg, company) [Default Company Ltd]:zfagaeva
Organizational Unit Name (eg, section) []:zfagaeva
Common Name (eg, your name or your server's hostname) []:zfagaeva.net
Email Address []:zfagaeva@zfagaeva.net
[root@server.zfagaeva.net private]# ls
localhost.key  www.zfagaeva.net.crt  www.zfagaeva.net.key
[root@server.zfagaeva.net private]# cp /etc/ssl/private/www.zfagaeva.net.crt /etc/ssl/certs/
[root@server.zfagaeva.net private]#
```

Рисунок 2.1: Генерация закрытого ключа и самоподписанного сертификата для веб-сервера

Для перевода виртуального веб-сервера `www.zfagaeva.net` на работу через HTTPS изменяю конфигурационный файл `/etc/httpd/conf.d/www.zfagaeva.net.conf` (см. рис. Рисунок 2.2). В файле создаю отдельные виртуальные хосты для портов 80 и 443. Первый виртуальный хост принимает обычные HTTP-запросы и перенаправляет их на HTTPS, а второй непосредственно обслуживает защищённые TLS-соединения.

Строка `<VirtualHost *:80>` открывает секцию виртуального хоста, принимающего соединения на TCP-порту 80 со всех сетевых интерфейсов сервера. Директива `ServerAdmin webmaster@zfagaeva.net` задаёт адрес администратора веб-сервера. Директива `DocumentRoot /var/www/html/www.zfagaeva.net` определяет каталог, содержащий файлы сайта. Строка `ServerName www.zfagaeva.net` задаёт основное доменное имя виртуального хоста, а `ServerAlias www.zfagaeva.net` определяет дополнительное имя, по которому Apache может сопоставить

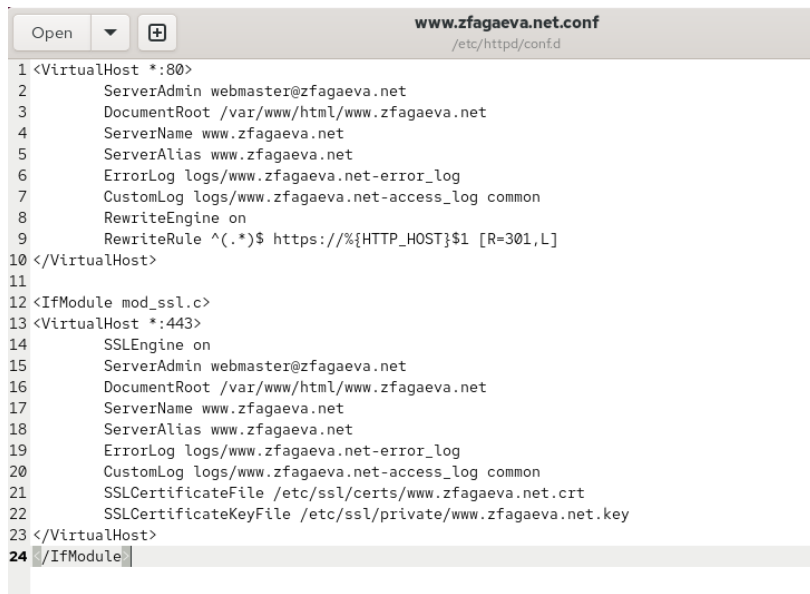
входящий запрос с данным виртуальным сервером.

Директива `ErrorLog logs/www.zfagaeva.net-error_log` задаёт файл журнала ошибок виртуального хоста. Директива `CustomLog logs/www.zfagaeva.net-access_log common` задаёт файл журнала обращений и указывает использовать стандартный формат журналирования `common`. Строка `RewriteEngine on` включает механизм преобразования URL модуля `mod_rewrite`. Правило `RewriteRule ^(.*)$ https://%{HTTP_HOST}$1 [R=301,L]` перехватывает входящий HTTP-запрос и формирует для него адрес с протоколом `https`. Переменная `%{HTTP_HOST}` содержит имя узла из HTTP-запроса, а `$1` содержит путь, соответствующий выражению `^(.*)$`. Флаг `R=301` выполняет постоянное перенаправление с кодом HTTP 301, а флаг `L` прекращает дальнейшую обработку правил после срабатывания данного правила. Строка `</VirtualHost>` завершает конфигурацию виртуального хоста на порту 80.

Строка `<IfModule mod_ssl.c>` указывает Apache обрабатывать расположенный внутри блок только при загруженном модуле `mod_ssl`. Далее директива `<VirtualHost *:443>` открывает конфигурацию виртуального хоста, принимающего HTTPS-соединения на стандартном TCP-порту 443. Строка `SSLEngine on` включает использование SSL/TLS для данного виртуального хоста.

Внутри HTTPS-виртуального хоста директива `ServerAdmin webmaster@zfagaeva.net` повторно задаёт адрес администратора, `DocumentRoot /var/www/html/www.zfagaeva.net` указывает каталог веб-содержимого, а `ServerName www.zfagaeva.net` и `ServerAlias www.zfagaeva.net` определяют доменное имя сервера. Строки `ErrorLog logs/www.zfagaeva.net-error_log` и `CustomLog logs/www.zfagaeva.net-access_log common` задают журналы ошибок и обращений.

Директива `SSLCertificateFile /etc/ssl/certs/www.zfagaeva.net.crt` задаёт путь к созданному сертификату X.509. Директива `SSLCertificateKeyFile /etc/ssl/private/www.zfagaeva.net.key` задаёт путь к соответствующему закрытому ключу. Строка `</VirtualHost>` завершает секцию HTTPS-виртуального хоста, а `</IfModule>` закрывает условный блок модуля `mod_ssl`.



```

1 <VirtualHost *:80>
2     ServerAdmin webmaster@zfagaeva.net
3     DocumentRoot /var/www/html/www.zfagaeva.net
4     ServerName www.zfagaeva.net
5     ServerAlias www.zfagaeva.net
6     ErrorLog logs/www.zfagaeva.net-error_log
7     CustomLog logs/www.zfagaeva.net-access_log common
8     RewriteEngine on
9     RewriteRule ^(.*)$ https://%{HTTP_HOST}$1 [R=301,L]
10 </VirtualHost>
11
12 <IfModule mod_ssl.c>
13 <VirtualHost *:443>
14     SSLEngine on
15     ServerAdmin webmaster@zfagaeva.net
16     DocumentRoot /var/www/html/www.zfagaeva.net
17     ServerName www.zfagaeva.net
18     ServerAlias www.zfagaeva.net
19     ErrorLog logs/www.zfagaeva.net-error_log
20     CustomLog logs/www.zfagaeva.net-access_log common
21     SSLCertificateFile /etc/ssl/certs/www.zfagaeva.net.crt
22     SSLCertificateKeyFile /etc/ssl/private/www.zfagaeva.net.key
23 </VirtualHost>
24 </IfModule>

```

Рисунок 2.2: Конфигурация виртуального хоста `www.zfagaeva.net` для автоматического перехода с HTTP на HTTPS

После настройки виртуального хоста изменяю параметры межсетевого экрана, разрешая входящие HTTPS-соединения. Добавляю сервис `https` в текущую конфигурацию командой `firewall-cmd --add-service=https`, после чего выполняю `firewall-cmd --add-service=https --permanent`, чтобы сохранить разрешение после перезагрузки системы. Обе команды завершаются сообщением `success`. Затем применяю постоянную конфигурацию командой `firewall-cmd --reload`, которая также выполняется успешно (см. рис. Рисунок 2.3).

После изменения межсетевого экрана перезапускаю Apache командой `systemctl restart httpd` и проверяю его состояние командой `systemctl status httpd`. Служба `httpd.service` находится в состоянии `active (running)`, а параметр `Loaded` показывает, что автоматический запуск службы включён. В сообщениях Apache присутствует строка `Started, listening on: port 443, port 80`, подтверждающая успешную загрузку конфигурации и одновременное прослушивание портов 443 и 80.

```

[root@server.zfagaeva.net conf.d]# firewall-cmd --add-service=https
success
[root@server.zfagaeva.net conf.d]# firewall-cmd --add-service=https --permanent
success
[root@server.zfagaeva.net conf.d]# firewall-cmd --reload
success
[root@server.zfagaeva.net conf.d]# systemctl restart httpd
[root@server.zfagaeva.net conf.d]# systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Sat 2026-09-26 07:46:36 UTC; 7s ago
     Invocation: 4fcf7b0fd81d4823b02c75e1da37b35b
       Docs: man:httpd.service(8)
    Main PID: 16748 (httpd)
      Status: "Started, listening on: port 443, port 80"
        Tasks: 178 (limit: 10230)
      Memory: 17.3M (peak: 17.6M)
         CPU: 45ms
    CGroup: /system.slice/httpd.service
            └─16748 /usr/sbin/httpd -DFOREGROUND
              └─16749 /usr/sbin/httpd -DFOREGROUND
                └─16750 /usr/sbin/httpd -DFOREGROUND
                  └─16751 /usr/sbin/httpd -DFOREGROUND
                    └─16752 /usr/sbin/httpd -DFOREGROUND
                      └─16753 /usr/sbin/httpd -DFOREGROUND

Sep 26 07:46:36 server.zfagaeva.net systemd[1]: Starting httpd.service - The Apache HTTP Server...
Sep 26 07:46:36 server.zfagaeva.net (httpd)[16748]: httpd.service: Referenced but unset environment variable evaluates
Sep 26 07:46:36 server.zfagaeva.net httpd[16748]: Server configured, listening on: port 443, port 80
Sep 26 07:46:36 server.zfagaeva.net systemd[1]: Started httpd.service - The Apache HTTP Server.
[root@server.zfagaeva.net conf.d]#

```

Рисунок 2.3: Разрешение HTTPS в межсетевом экране, перезапуск и проверка Apache HTTP Server

На виртуальной машине client открываю в браузере адрес www.zfagaeva.net. Настроенное правило перенаправления переводит запрос с HTTP на HTTPS, после чего Firefox выводит предупреждение о потенциальной угрозе безопасности (см. рис. Рисунок 2.4). Причиной предупреждения является использование самостоятельно подписанного сертификата, который не выдан доверенным центром сертификации. На странице указывается сообщение о недоверенном сертификате и код ошибки MOZILLA_PKIX_ERROR_SELF_SIGNED_CERT.

Для продолжения работы открываю дополнительные параметры предупреждения и выбираю продолжение перехода на сайт, принимая связанный с самоподписанным сертификатом риск.

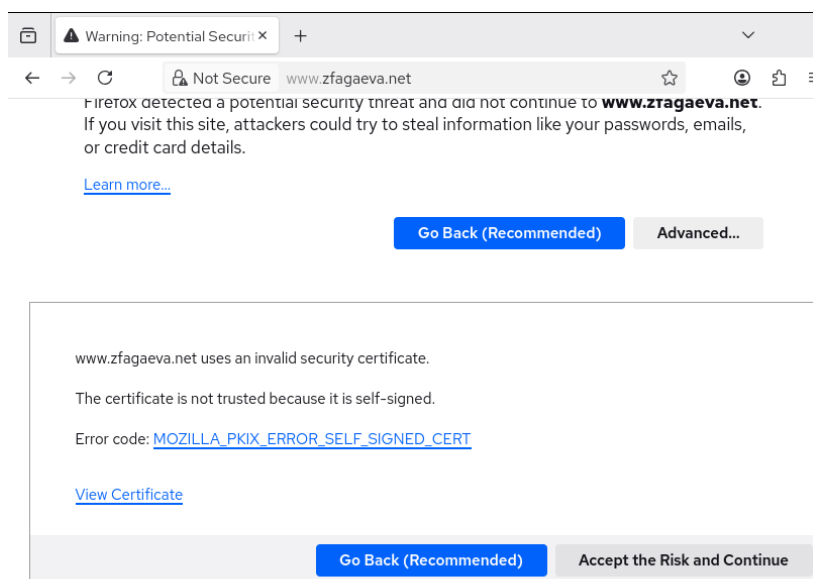


Рисунок 2.4: Предупреждение Firefox при первом подключении к серверу с самоподписанным сертификатом

После добавления исключения страница виртуального сервера `www.zfagaeva.net` успешно открывается по защищённому соединению (см. рис. Рисунок 2.5). В браузере отображается содержимое сайта с сообщением `Welcome to the www.zfagaeva.net server..` Это подтверждает работоспособность виртуального хоста после перехода на HTTPS и успешное использование Apache сертификата и закрытого ключа.



Рисунок 2.5: Открытие веб-сервера `www.zfagaeva.net` после перехода на HTTPS

В браузере просматриваю сведения об используемом сертификате (см. рис. Рисунок 2.6). В данных субъекта сертификата указаны страна RU, ре-

гион Russia, город Moscow, организация zfagaeva и подразделение zfagaeva. Поле Common Name содержит имя zfagaeva.net, а поле Email Address — адрес zfagaeva@zfagaeva.net. Выведенные значения соответствуют данным, указанным при создании сертификата средствами OpenSSL.

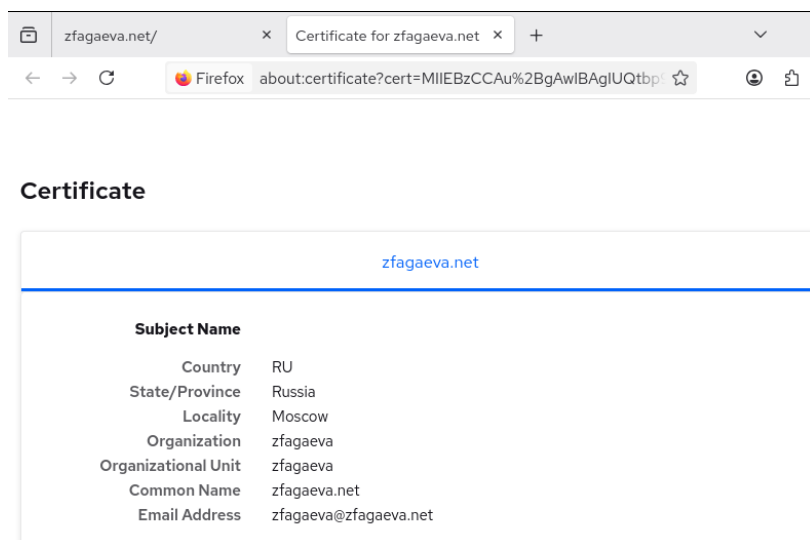


Рисунок 2.6: Просмотр сведений о самоподписанном сертификате веб-сервера

Для добавления поддержки PHP устанавливаю соответствующий пакет командой `dnf -y install php`. Вместе с PHP устанавливаются необходимые зависимости и компоненты, среди которых `php-cli`, `php-common`, `php-fpm`, `php-mbstring`, `php-opcache`, `php-pdo` и `php-xml`. В установленном наборе используется PHP версии 8.3.33.

После установки PHP изменяю главную страницу виртуального сервера. Файл `/var/www/html/www.zfagaeva.net/index.html` переименовываю в `/var/www/html/www.zfagaeva.net/index.php`, после чего открываю его для редактирования. В файле оставляю PHP-скрипт, вызывающий встроенную функцию `phpinfo()` (см. рис. Рисунок 2.7):

```
<?php
    phpinfo();
?>
```

Функция `phpinfo()` формирует диагностическую веб-страницу, содержащую сведения о версии PHP, параметрах интерпретатора, подключённых модулях, конфигурационных файлах и параметрах среды выполнения. Использование данного скрипта позволяет проверить, что PHP корректно обрабатывается веб-сервером, а исходный код PHP не передаётся клиенту как обычный текст.

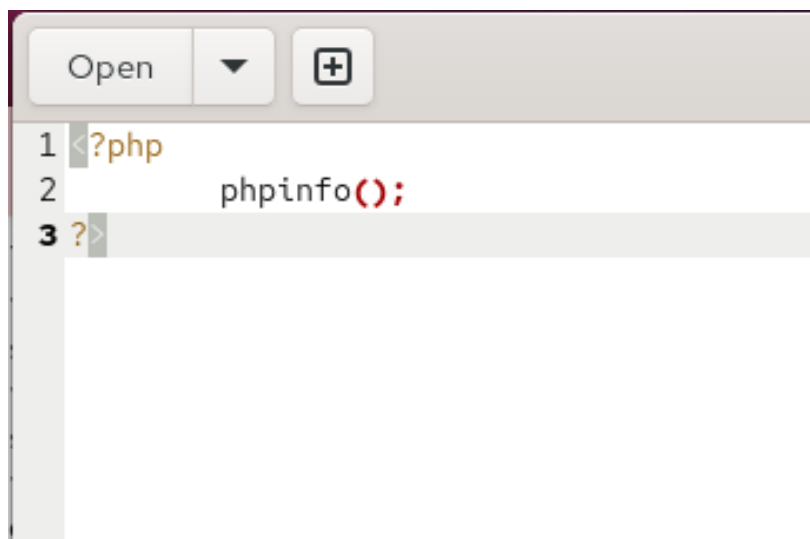


Рисунок 2.7: Содержимое файла `index.php` с вызовом функции `phpinfo`

Для корректного доступа Apache к веб-контенту изменяю владельца каталога `/var/www` и всех вложенных объектов командой `chown -R apache:apache /var/www`. Затем восстанавливаю стандартные контексты безопасности SELinux для системных конфигурационных файлов командой `restorecon -vR /etc` и для веб-контента командой `restorecon -vR /var/www` (см. рис. Рисунок 2.8). При выполнении `restorecon` система корректирует обнаруженные контексты файлов в соответствии с действующей политикой SELinux.

После изменения прав и контекстов безопасности перезапускаю веб-сервер командой `systemctl restart httpd`, чтобы Apache продолжил работу с установленной поддержкой PHP и обновлённым содержимым виртуального хоста.

```
Installed:
capstone-5.0.1-8.el10_2.x86_64      nginxfilesystem-2:1.26.3-6.el10_2.7.noarch  php-8.3.33-1.el10_2.x86_64
php-cli-8.3.33-1.el10_2.x86_64      php-common-8.3.33-1.el10_2.x86_64          php-fpm-8.3.33-1.el10_2.x86_64
php-mbstring-8.3.33-1.el10_2.x86_64  php-opcache-8.3.33-1.el10_2.x86_64         php-pdo-8.3.33-1.el10_2.x86_64
php-xml-8.3.33-1.el10_2.x86_64

Complete!
[root@server.zfagaeva.net conf.d]#
[root@server.zfagaeva.net conf.d]# mv /var/www/html/www.zfagaeva.net/index.html /var/www/html/www.zfagaeva.net/index.php
[root@server.zfagaeva.net conf.d]# gedit /var/www/html/www.zfagaeva.net/index.php
[root@server.zfagaeva.net conf.d]# chown -R apache:apache /var/www/
[root@server.zfagaeva.net conf.d]# restorecon -vR /etc
Relabeled /etc/NetworkManager/system-connections/eth1.nmconnection from unconfined_u:object_r:user_tmp_t:s0 to unconfined_u:object_r:NetworkManager_etc_rw_t:s0
[root@server.zfagaeva.net conf.d]# restorecon -vR /var/www/
[root@server.zfagaeva.net conf.d]# systemctl restart httpd
[root@server.zfagaeva.net conf.d]#
```

Рисунок 2.8: Установка PHP, настройка прав доступа, восстановление контекстов SELinux и перезапуск Apache

На виртуальной машине `client` повторно открываю `www.zfagaeva.net`. Вместо прежней статической страницы браузер отображает результат выполнения функции `phpinfo()` (см. рис. Рисунок 2.9). В верхней части страницы указана версия PHP 8.3.33. В разделе системной информации отображаются сведения о сервере `server.zfagaeva.net`, операционной системе Rocky Linux, архитектуре `x86_64`, интерфейсе веб-сервера FPM/FastCGI, расположении основного конфигурационного файла `/etc/php.ini` и каталоге дополнительных конфигурационных файлов `/etc/php.d`.

Появление страницы `phpinfo()` подтверждает, что запрос к файлу `index.php` обрабатывается интерпретатором PHP и результат выполнения скрипта успешно передаётся через Apache клиентскому браузеру.

PHP Version 8.3.33	
System	Linux server.zfagaeva.net 6.12.0-211.50.1.el10_2.x86_64 #1 SMP PREEMPT_DYNAMIC Tue 2026 x86_64
Build Date	Jul 28 2026 17:56:10
Build System	Rocky Linux release 10.2 (Red Quartz)
Build Provider	Rocky Enterprise Software Foundation
Compiler	gcc (GCC) 14.3.1 20251022 (Red Hat 14.3.1-4)
Architecture	x86_64
Server API	FPM/FastCGI
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc
Loaded Configuration File	/etc/php.ini
Scan this dir for additional .ini files	/etc/php.d
Additional .ini files parsed	/etc/php.d/10-opcache.ini, /etc/php.d/20-bz2.ini, /etc/php.d/20-calendar.ini, /etc/php.d/20-ctype.ini, /etc/php.d/20-curl.ini, /etc/php.d/20-dom.ini, /etc/php.d/20-exif.ini, /etc/php.d/20-fileinfo.ini, /etc/php.d/20-ftp.ini, /etc/php.d/20-gettext.ini, /etc/php.d/20-iconv.ini, /etc/php.d/20-mbstring.ini, /etc/php.d/20-phar.ini, /etc/php.d/20-simplexml.ini, /etc/php.d/20-sqlite3.ini, /etc/php.d/20-tokenizer.ini, /etc/php.d/20-xml.ini, /etc/php.d/20-xmlrpc.ini, /etc/php.d/20-zip.ini, /etc/php.d/20-zlib.ini

Рисунок 2.9: Проверка работы PHP на веб-сервере с помощью функции phpinfo

Для фиксации выполненной расширенной настройки веб-сервера подготавливаю provisioning-файлы Vagrant. Конфигурационные файлы Apache копирую из `/etc/httpd/conf.d` в каталог `/vagrant/provision/server/http/etc/httpd/conf.d`, а содержимое `/var/www/html` — в `/vagrant/provision/server/http/var/www/html`. Дополнительно создаю в каталоге provisioning структуру `/vagrant/provision/server/http/etc` и `/vagrant/provision/server/http/etc/pki/tls/certs`, предназначенную для хранения закрытого ключа и сертификата, после чего помещаю туда файлы `www.zfagaeva.net.key` и `www.zfagaeva.net.crt`.

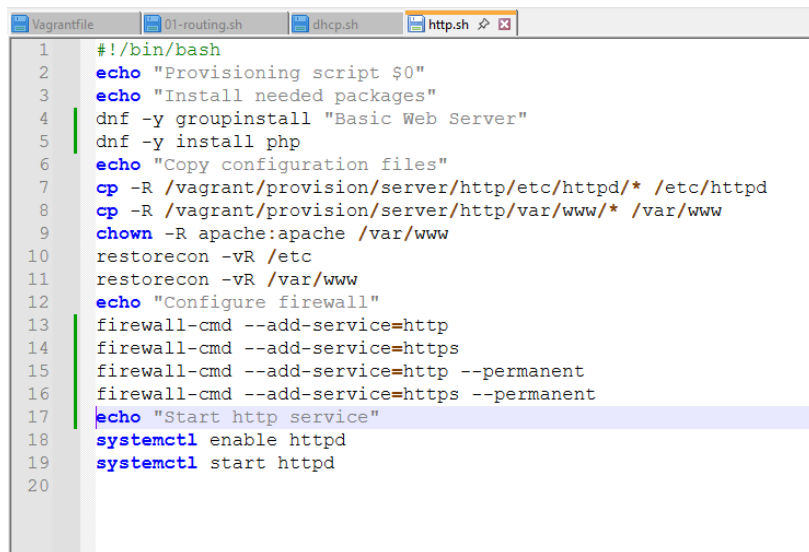
После сохранения конфигурационных файлов корректирую скрипт `/vagrant/provision/server/http.sh` (см. рис. Рисунок 2.10). В начале скрипта директива `#!/bin/bash` задаёт использование командной оболочки Bash. Команды `echo` выводят сообщения о выполняемых этапах настройки.

Команда `dnf -y groupinstall "Basic Web Server"` устанавливает стандартный набор программного обеспечения веб-сервера, а добавленная команда `dnf -y install php` обеспечивает автоматическую установку PHP при создании или повторном развёртывании виртуальной машины.

Команда `cp -R /vagrant/provision/server/http/etc/httpd/* /etc/httpd` переносит сохранённые конфигурационные файлы Apache из общего каталога Vagrant в системный каталог `/etc/httpd`. Команда `cp -R /vagrant/provision/server/http/var/www/* /var/www` восстанавливает подготовленное веб-содержимое. После копирования команда `chown -R apache:apache /var/www` назначает пользователю и группе `apache` владельцем файлов веб-сервера.

Команды `restorecon -vR /etc` и `restorecon -vR /var/www` восстанавливают корректные контексты SELinux для конфигурационных файлов и веб-контента. В разделе настройки межсетевого экрана команда `firewall-cmd --add-service=http` разрешает HTTP в текущей конфигурации, а `firewall-cmd --add-service=https` добавляет разрешение для HTTPS. Следующие команды `firewall-cmd --add-service=http --permanent` и `firewall-cmd --add-service=https --permanent` сохраняют оба разрешения в постоянной конфигурации межсетевого экрана.

В заключительной части скрипта команда `systemctl enable httpd` включает автоматический запуск Apache при загрузке операционной системы, а `systemctl start httpd` запускает службу веб-сервера. Благодаря добавлению установки `php` и разрешению сервиса `https` скрипт фиксирует основные изменения, выполненные при расширенной настройке HTTP-сервера, и позволяет автоматически воспроизводить их при развёртывании виртуальной машины `server`.

The image shows a Vagrant terminal window with four tabs: 'Vagrantfile', '01-routing.sh', 'dhcp.sh', and 'http.sh'. The 'http.sh' tab is active and displays a bash script. The script starts with a shebang and several echo statements for logging. It then uses 'dnf' to install 'Basic Web Server' and 'php'. Next, it copies configuration files from a Vagrant provision directory to the host's /etc/httpd and /var/www directories. It then uses 'restorecon' to set SELinux permissions on these directories. Finally, it configures the firewall to allow http and https traffic and starts the httpd service using 'systemctl'.

```
1  #!/bin/bash
2  echo "Provisioning script $0"
3  echo "Install needed packages"
4  dnf -y groupinstall "Basic Web Server"
5  dnf -y install php
6  echo "Copy configuration files"
7  cp -R /vagrant/provision/server/http/etc/httpd/* /etc/httpd
8  cp -R /vagrant/provision/server/http/var/www/* /var/www
9  chown -R apache:apache /var/www
10 restorecon -vR /etc
11 restorecon -vR /var/www
12 echo "Configure firewall"
13 firewall-cmd --add-service=http
14 firewall-cmd --add-service=https
15 firewall-cmd --add-service=http --permanent
16 firewall-cmd --add-service=https --permanent
17 echo "Start http service"
18 systemctl enable httpd
19 systemctl start httpd
20
```

Рисунок 2.10: Скорректированный provisioning-скрипт http.sh с установкой PHP и настройкой HTTP/HTTPS

3 Вывод

В ходе лабораторной работы была выполнена расширенная настройка HTTP-сервера Apache на виртуальной машине `server`. Для организации защищённого доступа к веб-серверу был сгенерирован закрытый RSA-ключ длиной 2048 бит и создан самоподписанный сертификат X.509 для доменного имени `www.zfagaeva.net`.

Конфигурация виртуального хоста `www.zfagaeva.net` была изменена для поддержки протокола HTTPS. Для HTTP-соединений на TCP-порту 80 было настроено автоматическое перенаправление запросов на HTTPS, а для защищённых соединений был настроен виртуальный хост на TCP-порту 443 с использованием созданных сертификата и закрытого ключа. В настройках межсетевого экрана был разрешён сервис `https`, после чего служба `httpd` была перезапущена. Работоспособность защищённого соединения проверена с виртуальной машины `client`.

Также была настроена поддержка языка PHP. На сервере установлены PHP и необходимые зависимости, а вместо статической страницы `index.html` создан файл `index.php` с вызовом функции `phpinfo()`. Для веб-контента были скорректированы права доступа и восстановлены контексты безопасности SELinux. Корректность настройки PHP подтверждена отображением в браузере страницы с информацией о версии и конфигурации PHP.

Для автоматизации расширенной настройки веб-сервера были сохранены конфигурационные файлы Apache, веб-контент, закрытый ключ и сертификат в каталоге `provisioning Vagrant`. Скрипт `/vagrant/provision/server/http.sh`

был дополнен установкой PHP и настройкой межсетевого экрана для разрешения соединений по протоколам HTTP и HTTPS. Это позволяет воспроизводить выполненную конфигурацию при автоматическом развёртывании виртуальной машины `server`.

4 Контрольные вопросы

1. В чём отличие HTTP от HTTPS?

HTTP (HyperText Transfer Protocol) — протокол прикладного уровня, используемый для передачи веб-страниц и других данных между клиентом и веб-сервером. При использовании обычного HTTP передаваемые данные не шифруются, поэтому при возможности перехвата сетевого трафика их содержимое потенциально может быть прочитано или изменено третьей стороной.

HTTPS (HyperText Transfer Protocol Secure) представляет собой HTTP, работающий поверх защищённого соединения TLS. Перед передачей HTTP-данных клиент и сервер устанавливают защищённое соединение, после чего передаваемая между ними информация шифруется.

По умолчанию HTTP использует TCP-порт 80, а HTTPS — TCP-порт 443.

Основное отличие заключается в том, что HTTPS обеспечивает шифрование передаваемых данных, проверку их целостности и возможность проверки подлинности веб-сервера с помощью цифрового сертификата.

2. Каким образом обеспечивается безопасность контента веб-сервера при работе через HTTPS?

Безопасность обмена данными при использовании HTTPS обеспечивается протоколом TLS и применением криптографических механизмов.

При подключении клиента к HTTPS-серверу веб-сервер предоставляет свой цифровой сертификат. Сертификат содержит сведения о сервере и

его открытый ключ. Клиент проверяет сертификат, после чего в процессе TLS-рукопожатия клиент и сервер согласовывают криптографические параметры и формируют сеансовые ключи.

После установления соединения передаваемые HTTP-запросы и ответы шифруются с использованием сеансовых ключей. Благодаря этому перехватившая сетевой трафик третья сторона не может непосредственно прочитать передаваемое содержимое.

HTTPS обеспечивает три основных свойства безопасности:

- **конфиденциальность** — данные между клиентом и сервером передаются в зашифрованном виде;
- **целостность** — TLS позволяет обнаружить изменение передаваемой информации во время её передачи;
- **аутентификацию сервера** — цифровой сертификат позволяет клиенту проверить, что соединение установлено с сервером, которому принадлежит предъявленный сертификат.

В лабораторной работе использовался самоподписанный сертификат. Он позволяет создать зашифрованное HTTPS-соединение, однако браузер не может автоматически подтвердить доверие к его издателю, поэтому при первом подключении отображается предупреждение о недоверенном сертификате.

3. Что такое сертификационный центр? Приведите пример.

Сертификационный центр, или CA (Certificate Authority), — организация или служба, занимающаяся выпуском и подтверждением цифровых сертификатов. Сертификационный центр проверяет сведения о владельце сертификата в соответствии со своей процедурой проверки и подписывает сертификат собственной цифровой подписью.

Браузеры и операционные системы содержат хранилища доверенных корневых сертификатов сертификационных центров. Если сертификат

веб-сервера связан с доверенным сертификационным центром корректной цепочкой сертификатов и остальные проверки успешно пройдены, браузер может установить доверенное HTTPS-соединение без предупреждения о неизвестном издателе.

Примером сертификационного центра является Let's Encrypt. Он предоставляет бесплатные TLS-сертификаты и поддерживает автоматизированный процесс их получения и обновления. Сертификаты Let's Encrypt широко используются для организации защищённого доступа к веб-сайтам по протоколу HTTPS.